

Competitive Programming

Section 29: Modular Arithmetic

Competitive Programming Course

2026-07-10

Table of contents

1	Why this matters under contest pressure	3
2	Vocabulary and the arithmetic contract	4
3	Recognition and limits	5
4	Binary exponentiation	6
4.1	Complete trace: $3^{13} \bmod 17$	6
4.2	Proof habit	6
5	Modular inverse and fraction queries	7
5.1	Complete trace: $10 / 4 \bmod 17$	7
6	Repeated affine transformations	8
6.1	Complete trace: the <code>a == 1</code> branch	8
7	Implementation notes	9
7.1	C++	9
7.2	Python	9
8	Wrong approaches and targeted tests	10
9	Formative exercises	11
10	Local workflow and submission checklist	12
11	Exercises	13

1 Why this matters under contest pressure

Modulo is not an instruction to append `% MOD` to a formula. It is a number system in which values are represented by remainders. In contests, it appears when a count is too large to print, a recurrence runs for an enormous number of steps, or a probability must be represented exactly rather than as a decimal.

The useful first question is not “what is the modulus?” It is “which operation is too large or illegal in ordinary arithmetic?”

Statement signal	First tool	Local problem
exponent up to 10^{18}	binary exponentiation	A
quotient modulo a stated prime	modular inverse	B
repeat <code>x <- a*x+b</code> many times	powers plus a geometric sum	C

2 Vocabulary and the arithmetic contract

For a positive modulus M , $x \bmod M$ means the unique representative in $0..M-1$ congruent to x . Addition and multiplication may be reduced at any time:

```
(u + v) mod M = ((u mod M) + (v mod M)) mod M
(u * v) mod M = ((u mod M) * (v mod M)) mod M
```

Subtraction needs normalization. Python already returns a nonnegative result:

```
(-1) % 7 == 6
```

In C++, the remainder keeps the left operand's sign, so normalize explicitly:

```
long long norm(long long x, long long mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
```

Division is different. $a / b \bmod M$ means $a * \text{inv}(b) \bmod M$, where $b * \text{inv}(b) == 1 \pmod{M}$. Such an inverse exists exactly when $\text{gcd}(b, M) = 1$. For this section's fraction problem, M is prime and b is nonzero modulo M , so Fermat's theorem supplies $\text{inv}(b) = b^{M-2} \bmod M$. Section 30 handles the general composite-modulus case with extended Euclid.

3 Recognition and limits

Read constraints before choosing a formula.

```
Exponent  $10^{18}$ : an  $O(\text{exponent})$  loop is impossible; use  $O(\log \text{exponent})$ .  
Prime modulus plus a nonzero denominator: Fermat inverse is legal.  
Composite modulus: do not use  $M-2$ ; first check gcd and use extended Euclid.  
Repeated affine update: unroll three steps before reaching for a matrix.
```

For the local exercises, products of two reduced values fit in signed `long long` when the modulus is 1,000,000,007. A reusable routine for a modulus near 10^{18} needs `__int128` in C++:

```
long long mul_mod(long long a, long long b, long long mod) {  
    return (long long)((__int128)a * b % mod);  
}
```

Python integers do not overflow, but an enormous intermediate power is still too slow and memory-hungry. Use `pow(a, b, mod)` or the binary loop.

4 Binary exponentiation

Write the exponent in binary. For example, $13 = 8+4+1$, so $a^{13} = a^8 \cdot a^4 \cdot a$. The loop stores the current power of two in `base` and moves selected powers into `result`.

```
result = 1 mod M
base = a mod M
while exponent > 0:
    if exponent is odd:
        result = result * base mod M
    base = base * base mod M
    exponent = floor(exponent / 2)
```

4.1 Complete trace: $3^{13} \bmod 17$

The invariant is `result * baseexponent == 313 (mod 17)`.

result	base	exponent	action
1	3	13	odd: result = 3
3	9	6	square, halve
3	13	3	odd: result = 5; square, halve
5	16	1	odd: result = 12; square, halve
12	1	0	stop

The answer is 12; indeed $3^{13} = 1594323$, whose remainder modulo 17 is 12. Every iteration either consumes one binary bit or halves the remaining exponent, so there are only $O(\log \text{exponent})$ iterations.

4.2 Proof habit

Before coding, write the invariant using the original base and exponent:

```
result * baseremaining == original_baseoriginal_exponent (mod M).
```

On an odd iteration, moving one `base` into `result` leaves an even exponent. For an even exponent, replacing `basee` by `(base2)(e/2)` preserves the product. At exponent zero, the invariant says `result` is the answer.

5 Modular inverse and fraction queries

Under prime P , Fermat says $b^{(P-1)} \equiv 1 \pmod{P}$ whenever $b \bmod P \neq 0$. Thus $b^{(P-2)}$ is the multiplicative inverse of b .

5.1 Complete trace: $10 / 4 \bmod 17$

Binary exponentiation gives $4^{15} \bmod 17 = 13$. Check it before trusting it:

```
4 * 13 = 52 = 1 (mod 17).  
10 / 4 = 10 * 13 = 130 = 11 (mod 17).
```

The final verification is $4 * 11 \equiv 10 \pmod{17}$. Ordinary integer division would give 2, which is unrelated to modular division.

The legality checklist is short and strict:

```
Is the modulus prime?  
Is b nonzero modulo that prime?  
If either answer is no, do not use  $b^{(P-2)}$ .
```

6 Repeated affine transformations

For $f(x)=ax+b$, expand a few applications:

```
f^1(x) = a*x + b
f^2(x) = a^2*x + a*b + b
f^3(x) = a^3*x + a^2*b + a*b + b
```

After k applications,

```
f^k(x) = a^k*x + b*(1+a+...+a^(k-1)).
```

When $a \neq 1$, the sum is $(a^k-1)/(a-1)$. When $a=1$, every term is one, so the sum is k . Check $a=1$ only *after* reducing it modulo MOD.

6.1 Complete trace: the $a == 1$ branch

Let $f(x)=x+3$, $x=5$, and $k=4$ modulo 17.

```
direct: 5 -> 8 -> 11 -> 14 -> 0
formula: 1^4*5 + 3*(1+1+1+1) = 5+12 = 17 = 0 mod 17
```

Trying $(a^k-1)/(a-1)$ here would divide by zero. This is not an implementation annoyance: the mathematical formula has a distinct case.

7 Implementation notes

7.1 C++

Use `long long` for exponents and values. In a generic-power routine, start with `result = 1 % mod` so `mod=1` correctly returns zero even for exponent zero. Normalize all potentially negative affine inputs before comparing `a==1`. Use `__int128` for multiplication if the statement permits moduli large enough that two residues can overflow `long long`.

7.2 Python

Use `pow(base, exponent, mod)` for a compact optimized exponentiation and `sys.stdin.buffer` for many queries. Python's `%` already normalizes negative values, but normalize `a`, `b`, and `x` before the affine branch so the code states the mathematical invariant clearly.

8 Wrong approaches and targeted tests

Tempting approach	Why it fails	Test that exposes it
multiply a by itself b times initialize result to 1 unconditionally	b can be 10^{18} $1 \bmod 1$ is zero	A with $b=10^{18}$ A with $b=0, m=1$
use ordinary / or floating point	modular division needs an inverse	B: $10/4 \bmod 17$
use Fermat for a composite modulus	$b^{(M-2)}$ need not be inverse	future variant: $6 \bmod 35$
use geometric division when $a=1$	denominator is zero	C: $a=1, k=4$
compare unreduced a with one	$a=\text{MOD}+1$ is the same residue	C: $a=1000000008$

9 Formative exercises

1. Trace binary exponentiation for $2^{11} \bmod 13$. Record **result**, **base**, and remaining exponent after every iteration.
2. Compute $7/3 \bmod 11$ and verify the result by multiplying it by three.
3. Let $f(x)=x+5$ modulo 19. Find $f^7(4)$ without simulating seven updates.
4. Explain why $6/10 \bmod 21$ is not a legal Fermat-inverse question, even though it resembles problem B.

Answers:

1. $2^{11} \bmod 13 = 7$. The invariant, not a memorized table, is the check.
2. $\text{inverse}(3)=4$, so $7/3 = 28 = 6 \bmod 11$; $3*6 = 7 \bmod 11$.
3. $4 + 7*5 = 39 = 1 \bmod 19$.
4. 21 is composite and $\text{gcd}(10,21)=1$ only licenses extended Euclid, not Fermat.

10 Local workflow and submission checklist

Run the complete local suite:

```
CP_TARGET=solution python3 sections/29_modular_arithmetic/check.py
```

For A, compare the fast routine with a slow loop on small exponents. For B, verify every output by multiplying it by the denominator modulo P . For C, compare the closed form with direct simulation for small k .

1. Did I establish that every modular inverse exists?
2. Is every exponent loop $O(\log \text{exponent})$?
3. Did I normalize C++ negative residues?
4. Did I test modulus one, exponent zero, and affine $a=1$?
5. Does each multiplication fit its language's integer type?

11 Exercises

Solve local A after completing exercise 1, local B after exercise 2, and local C after exercise 3. Before submitting each one, add its targeted edge cases to your personal test input and state the invariant or proof idea in one sentence.